🥋 DAY 1: KANTHAST MASTER TRAINER WAR-ROOM RUNBOOK & SYNC MAP

Complete Step-by-Step Delivery Playbook, Presentation Sync, MCQ Injection & Hands-On Setup

Institution: Department of Computer Engineering & Applications, GLA University, Mathura (AB12 Building)
Alliance: Intel & NEC Supercomputing Alliance × Edulateral Foundation
Lead Trainer: Shivam Gupta | **Co-Trainer:** Co-Trainer (Morning Theory Lead / Afternoon Lab Co-Pilot)
Coordinator: Pedina Sreeja (GLA University Training Helper)
Official Curriculum Reference: `Intel & NEC - High Performance Computing.pdf` (Page 2)
Master Presentation Deck: `Day-01_Master_Presentation.pptx` | `Day-01_Edulateral_Official.pptx`
Google Colab Lab Notebook: `Day_01_Intel_oneAPI_and_HPC_Toolkit_Master_Lab.ipynb`

🎯 EXECUTIVE GROUND TRUTH & DAY 1 BOUNDARIES

```
+-------------------------------------------------------------------------------+
|                          STRICT SYLLABUS BOUNDARY INVARIANT                   |
+-------------------------------------------------------------------------------+
|  IN SCOPE TODAY (DAY 1 ONLY):                                                 |
|  1. Introduction to HPC (Why laptops fail, Memory Wall, FLOPS, TOP500)        |
|  2. Intel HPC Architecture (Xeon Scalable CPUs, Max Series GPUs, Habana Gaudi 2/3)
|  3. Linux Basics (Filesystem Hierarchy, Absolute vs Relative, Core CLI, $PATH)
|  4. Shell Scripting for Automation (Shebang, Variables, Redirection, hpc_node_audit.
|  5. Secured Shell (SSH) (Asymmetric Keys, ed25519, authorized_keys, remote demo)
|  6. Intel oneAPI Toolkit (Unified Cross-Architecture Model, SYCL, setvars.sh, sycl-l
|  7. Introduction to Intel HPC Toolkit (icx/icpx, ifx, Intel MPI runtime simulation)
+-------------------------------------------------------------------------------+
|  STRICTLY OUT OF SCOPE TODAY (DO NOT PRE-EMPT):                               |
|  • Day 2: OpenMP multithreading, MPI distributed algorithms, oneDAL machine learning
|  • Day 3: Deep Learning on HPC, TensorFlow-MKL, oneDNN, Slurm batch job scheduler
```

```
|   • Day 4: Distributed AI training with Horovod, Intel VTune performance profiling
|   • Day 5: Generative AI, LLM fine-tuning on Habana Gaudi, OpenVINO model inference
+-------------------------------------------------------------------------------------
```

⏰ THE GRAND OPERATIONAL TIMELINE (10:00 AM – 05:00 PM)

Synchronized with Pedina Sreeja's WhatsApp schedule (Car pickup < 10:00 AM from Guest House to AB12; Pre-assessment @ 10:30–10:50 AM)

```
+-------------------------------------------------------------------------------------
|                          DAY 1 MASTER TIMELINE MAP
+-------------------------+-----------------------------------------------------------
| 09:30 AM - 10:00 AM     | Guest House to AB12 Transit, Lab Preflight, Projector & W
+-------------------------+-----------------------------------------------------------
| 👱 MORNING SESSION      | LEAD: CO-TRAINER | CO-PILOT & CLUSTER DEMO: SHIVAM GUPTA
| 10:00 AM - 10:30 AM (30m)| PART 1: Welcome, Program Pedagogy & Scope (Slides 1 - 6)
| 10:30 AM - 10:50 AM (20m)| 📝 OFFICIAL 50-MCQ PRE-ASSESSMENT (Administered with GLA
| 10:50 AM - 11:35 AM (45m)| PART 2: Intro to HPC & Intel Architecture (Slides 7 - 9)
| 11:35 AM - 11:45 AM (10m)| ☕ Bio & Rejuvenation Break
| 11:45 AM - 12:35 PM (50m)| PART 3: Intel oneAPI & HPC Toolkit Ecosystem (Slides 12 -
| 12:35 PM - 12:45 PM (10m)| PART 4: 5 Socratic Checkpoint MCQs & Lunch Teaser (Slide
+-------------------------+-----------------------------------------------------------
| 🍱 12:45 PM - 01:45 PM   | LUNCH BREAK (60 Mins) — Cluster Ping & Student Account A
+-------------------------+-----------------------------------------------------------
| 💻 AFTERNOON SESSION    | LEAD: SHIVAM GUPTA | LAB SUPPORT: CO-TRAINER
| 01:45 PM - 02:30 PM (45m)| STAGE 0 & 1: System Readiness & Silicon Introspection (ls
| 02:30 PM - 03:15 PM (45m)| STAGE 2 & 3: Linux Tree, Heredocs, Permissions & $PATH
| 03:15 PM - 04:00 PM (45m)| STAGE 4: Automated Shell Scripting (hpc_node_audit.sh Dri
| 04:00 PM - 04:30 PM (30m)| STAGE 5 & 6: SSH Key Generation (ed25519) & oneAPI Setup
| 04:30 PM - 04:50 PM (20m)| 🎯 25 IN-CLASS CHECKPOINT MCQS & 22-POINT CHECKLIST SIGN
| 04:50 PM - 05:00 PM (10m)| Day 1 Wrap-up, Q&A & Day 2 Teaser (OpenMP / MPI Preview)
+-------------------------+-----------------------------------------------------------
```

🧠 THE "KANTHASTH" 7-PILLAR MEMORIZATION ENGINE

(Every trainer must know these 7 core pillars by heart — no hesitation, instant recall)

1. HPC Core Definition & The Memory Wall

- **What is HPC?** Aggregating compute nodes across high-speed fabrics to deliver computational throughput impossible on single workstations.
- **Why laptops fail:** Not because CPU clock is too slow, but because of the **Memory Wall**:
- A high-end laptop Core i7/i9 has **2 DDR memory channels** (~50–80 GB/s bandwidth).
- An enterprise Dual-Socket Intel Xeon Scalable node (Sapphire Rapids) has **16 DDR5 memory channels** (>600 GB/s bandwidth).
- When training models or computing large matrices, the laptop CPU spends >85% of cycles stalled waiting for RAM. HPC feeds the ALUs continuously.
- **Key Metric:** FLOPS (Floating-point Operations Per Second) benchmarked via **HPL (High Performance LINPACK)** for the TOP500 list.

2. The Intel Hardware Triad

- **Intel Xeon Scalable CPUs (Sapphire Rapids / Emerald Rapids):**
- High-frequency sequential logic, OS orchestration, data preprocessing, graph traversals, FP64 scientific math.
- Built-in hardware engine: **Intel AMX (Advanced Matrix Extensions)** with 8 2D tile registers (tmm0-tmm7, 8 KB total) for on-die matrix multiply acceleration.
- **Intel Data Center GPUs (Max Series / Ponte Vecchio):**
- Thousands of lightweight SIMD vector cores across Xe-Cores.
- Built for massive FP32/FP16 matrix math and parallel simulations.
- **Habana Gaudi AI Accelerators (Gaudi 2 & Gaudi 3):**
- Purpose-built from silicon up exclusively for deep learning and LLM training.
- Features 24 Tensor Processor Cores (TPCs) and **on-die integrated 24x 100GbE RoCE RDMA networking (300 GB/s aggregate throughput)** eliminating external NIC bottlenecks.

3. Linux Fundamentals & VFS Invariants

- **Golden Rule:** In UNIX/Linux, *Everything is a file* arranged in a single hierarchical tree starting at root `/`.
- **Paths:** Absolute starts with `/` (`/home/user/script.sh`); Relative starts from current directory (`./script.sh` , `../data`).
- **HPC Storage Hierarchy:**
- `/home` : "Cold Zone" — Networked NFS storage, quota-limited, ideal for source code and scripts.
- `/scratch` or `/work` : "Hot Zone" — High-speed parallel NVMe storage (Lustre/GPFS), ephemeral, required for dataset storage and checkpoints.

- **Environment & Search:** `$PATH` is a colon-separated list of directories searched left-to-right when commands are typed without paths.

4. Shell Automation & Robust Scripting

- **Shebang:** `#!/bin/bash` or `#!/usr/bin/env bash` on Line 1 directs the kernel `execve()` system call to the Bash interpreter.
- **Variable Syntax:** `VAR="value"` — **NO SPACES around** `=` ! Bash interprets spaces as command arguments.
- **Defensive Bash Directive:** `set -euo pipefail`
- `-e` : Abort immediately if any command exits non-zero.
- `-u` : Treat unset variables as an error and abort.
- `-o pipefail` : Trap errors inside piped commands (e.g., `cmd1 | cmd2`).
- **Output Redirection:** `>` (overwrite), `>>` (append), `2>&1` (redirect stderr to stdout).

5. Secured Shell (SSH) & Asymmetric Cryptography

- **Protocol:** Encrypted terminal connection over TCP Port 22.
- **Password vs Key-Based:**
- Password: Slow, brute-forceable, cannot be automated in batch schedulers.
- Key-based: Asymmetric cryptography using **Ed25519** elliptic curve keys.
- **Key Mechanics:**
- `id_ed25519` (Private Key): Stored locally, **permissions strictly 600** (`chmod 600 ~/.ssh/id_ed25519`). NEVER shared!
- `id_ed25519.pub` (Public Key): Appended to remote node's `~/.ssh/authorized_keys` .
- Server challenges client with encrypted nonce; client proves possession of private key without sending it across the wire.

6. Intel oneAPI Unified Architecture

- **The Problem:** Proprietary vendor lock-in (CUDA locked to NVIDIA, rewriting for OpenCL).
- **The Solution:** Intel oneAPI is an open, standards-based, cross-architecture programming model built on Khronos **SYCL** (Standard C++ heterogeneous abstraction).
- **Single-Source Vision:** Write clean modern C++ once; compile and execute across Intel/NVIDIA/AMD CPUs, GPUs, and FPGAs.
- **Activation Command:** `source /opt/intel/oneapi/setvars.sh`
- **Device Inspection:** `sycl-ls` lists all available heterogeneous compute targets.

7. Intel HPC Toolkit Overview

- **Base Toolkit vs HPC Toolkit:**
- *Base Toolkit:* The Single-Node Heterogeneous Foundation (DPC++/SYCL, oneMKL, oneDNN, oneDAL).
- *HPC Toolkit:* The Cluster-Scale Supercomputing Booster (Intel C/C++ Compilers `icx` / `icpx`, Intel Fortran `ifx`, Intel MPI Library, Intel Inspector).
- **HPC Lifecycle:** C++/SYCL Code ──► Compile with `icpx` / `mpicxx` ──► Profile / Debug ──► Cluster Batch Launch.

📽️ PRESENTATION SYNC: SLIDE-BY-SLIDE DELIVERY SCRIPT (SLIDES 1 – 15)

Source Presentations: `Day-01_Master_Presentation.pptx` & `Day_01_Slide_OCR_Transcript.md`

```
+-------------------------------------------------------------------------------------------------------------------------------+
|                                        SLIDE-BY-SLIDE TRAINER DELIVERY & CLASSROOM ACTIONS                                    |
+-------------------------------------------------------------------------------------------------------------------------------+
```

🔷 SLIDE 1: High Performance Computing: A Practical Approach

- **Time:** 10:00 AM – 10:05 AM (5 Mins) | **Speaker:** Co-Trainer (Welcome) + Shivam (Context)
- **What to Show:** Title slide with Intel & NEC Supercomputing Alliance branding.
- **Verbatim Trainer Speech (Hinglish/English):**

> "Good morning everyone! Welcome to Day 1 of our intensive High Performance Computing and Heterogeneous AI Systems masterclass at GLA University. Over the next 5 days, we are going to bridge the fundamental gap between writing hobbyist code on personal laptops and engineering high-throughput, cluster-scale applications. Today is Day 1. By 5:00 PM today, every single student in this room will know how supercomputers work, how Intel silicon operates, how to navigate Linux like a systems engineer, how to automate cluster tasks using Bash scripts, how to connect securely via SSH, and how to initialize the Intel oneAPI ecosystem."

- **Classroom Action:** Instruct all 60 students to open their laptops, ensure Wi-Fi connectivity, and bookmark the Day 1 Google Colab notebook link.

🔷 SLIDE 2: AI Engineering: From Workstation to HPC Cluster

- **Time:** 10:05 AM – 10:12 AM (7 Mins) | **Speaker:** Co-Trainer
- **What to Show:** Comparison diagram: Standard Single-Node Laptop vs Distributed HPC Cluster.
- **Verbatim Trainer Speech:**

> "Look at your laptop right now. If you train a model on 10,000 rows, it works. But what happens when you train on 500 million rows or a 70-Billion parameter LLM? First, your RAM runs out. Second, your memory bandwidth is choked. Third, your CPU thermal-throttles and crashes. In HPC, we don't just buy a 100x bigger laptop — we connect dozens of enterprise compute nodes using high-speed interconnects and shared parallel filesystems. You are transitioning today from writing baseline scripts to engineering optimized, cluster-scale AI."

- **Whiteboard Diagram (Draw this on physical board):** `[Personal Laptop] [HPC Supercomputer Cluster] +-----------------+ +-----------+ +-----------+ | CPU (8 Cores) | | Head Node | | Compute 01 | | RAM: 16 GB | +-----+------+ +-----+------+ | 2 Memory Ch. | | (InfiniBand 100Gbps) | | ~60 GB/s BW | +-----+------+ +-----+------+ +-----------------+ | Compute 02 | ... | Compute 64 | (Memory Wall Choke) +-----------+ +-----------+ (Shared Parallel Storage: Lustre/GPFS)`

🔷 SLIDE 3: Our Pedagogical Engine

- **Time:** 10:12 AM – 10:18 AM (6 Mins) | **Speaker:** Co-Trainer
- **What to Show:** 3-Step Pedagogical Loop: `[01] Concept ⟶ [02] System Understanding ⟶ [03] Hands-On`.
- **Verbatim Trainer Speech:**

> "We follow a strict 3-step discipline for every topic: First: **Concept**. Understand the mathematical problem and parallel theory. Second: **System Understanding**. Understand how physical silicon — Intel Xeon CPUs, GPUs, and Gaudi accelerators — actually executes the instructions in hardware registers and cache lines. Third: **Hands-On**. Open your terminal and run verified code on bare-metal. No hand-waving, no placeholders."

🔷 SLIDE 4: The Core Pillars of Training

- **Time:** 10:18 AM – 10:24 AM (6 Mins) | **Speaker:** Co-Trainer
- **What to Show:** 4 Pillar Cards: Industry-Grade Tools, Performance Focus, Cluster-Based Learning, Project-Oriented.
- **Verbatim Trainer Speech:**

> "This workshop is built on genuine industry tooling: Intel oneAPI, oneMKL, oneDNN, Slurm, VTune, and Habana Gaudi. Everything we do compounds into real supercomputing workflows."

🔷 SLIDE 5: The Software & Hardware Ecosystem

- **Time:** 10:24 AM – 10:27 AM (3 Mins) | **Speaker:** Co-Trainer
- **What to Show:** Layered stack: Compute & Silicon ──► Orchestration & Access ──► Execution & Optimization.
- **Verbatim Trainer Speech:**

> "Notice how the layers connect: At the bottom is Silicon (Xeon, Max GPU, Gaudi). Above that is the OS and Network (Linux, SSH). In the middle is the bridge: Intel oneAPI. And at the top are your applications."

🔷 SLIDE 6: The 5-Day Curriculum Arc

- **Time:** 10:27 AM – 10:30 AM (3 Mins) | **Speaker:** Co-Trainer + Shivam
- **What to Show:** 5-Day roadmap showing: `Day 1 (You Are Here)` to `Day 5 (Generative AI on Gaudi)`.
- **Verbatim Trainer Speech:**

> "Look at our 5-day progression: - **Day 1 (Today):** Fundamentals of HPC, Linux, Shell & Toolkits. - **Day 2:** Traditional ML using Intel oneAPI, OpenMP, and MPI. - **Day 3:** Deep Learning on HPC with TensorFlow-MKL, GPUs, and Slurm. - **Day 4:** Distributed AI Training & VTune Profiling. - **Day 5:** Generative AI & LLMs on Intel HPC hardware (Gaudi, OpenVINO). We respect our boundaries: Today, we master Day 1 completely before moving forward."

📝 OFFICIAL PRE-ASSESSMENT (10:30 AM – 10:50 AM | 20 MINS)

- **Location in Workspace:** `quizzes_and_assessments/pre_assessment/`
- **Student Paper:** `GLA_UNIVERSITY_DAY_01_PRE_ASSESSMENT_STUDENT_QUESTION_PAPER.md` | PDF
- **Instructor Key:** `GLA_UNIVERSITY_DAY_01_50_MCQ_PRE_ASSESSMENT_OFFICIAL.md` | Excel
- **Trainer Announcement (10:30 AM Sharp):**

> "Attention students: As per the GLA University schedule coordinated with Pedina Sreeja ma'am, we are now conducting the official Day 1 Pre-Assessment from 10:30 AM to 10:50 AM. Please open the assessment link on your portal (or question paper). You have exactly 20 minutes. Do not worry if you encounter unfamiliar terms like AMX, RoCE, or SYCL — this is a diagnostic benchmark to establish your baseline before today's training begins!"

- **Trainer Action:** Co-Trainer and Shivam walk the lab room, ensuring all 60 students are logged in and submitting.

🔷 SLIDE 7: Agenda: `> init --module day_01_fundamentals`

- **Time:** 10:50 AM – 10:55 AM (5 Mins) | **Speaker:** Co-Trainer
- **What to Show:** Terminal status screen displaying Day 1 modules.
- **Verbatim Trainer Speech:**

> "With pre-assessment complete, let's load our modules: HPC Architecture, The Hardware Continuum, Linux OS Navigation, SSH & Shell Automation, and Toolkit Initialization."

🔷 SLIDE 8: Deconstructing HPC: Beyond the Laptop

- **Time:** 10:55 AM – 11:15 AM (20 Mins) | **Speaker:** Co-Trainer
- **What to Show:** Detailed hardware comparison: Memory Channels, NUMA, Interconnects.
- **Verbatim Trainer Speech:**

> "Why does an ordinary laptop choke on large AI workloads? The primary culprit is the **Memory Wall**. An Intel Core i7 laptop has 2 DDR memory channels giving ~50 to 80 GB/s bandwidth. An enterprise Dual-Socket Intel Xeon Scalable server has **16 DDR5 memory channels giving over 600 GB/s bandwidth**! Furthermore, HPC nodes use ECC (Error-Correcting Code) RAM so that cosmic rays or voltage blips do not flip bits during a 3-week simulation run. Supercomputers are ranked on the TOP500 list using High Performance LINPACK (HPL), measured in FLOPS."

- **Classroom Check (Ask Class):** "*What is the peak theoretical FLOPS formula?*" *(Answer: Sockets × Cores/Socket × Clock Speed × FLOPS/cycle).*

🔷 SLIDE 9: The Hardware Continuum: Xeon, GPUs & Gaudi

- **Time:** 11:15 AM – 11:35 AM (20 Mins) | **Speaker:** Co-Trainer + Shivam

- **What to Show:** Three-column table: Intel Xeon CPUs vs Intel Data Center GPUs vs Habana Gaudi.
- **Verbatim Trainer Speech:**

> "Modern supercomputers are heterogeneous. There is no single chip that does everything best: 1. **Intel Xeon Scalable CPUs:** The orchestrators. High single-thread clock speed, massive L3 caches, AVX-512 vector units, and **Intel AMX (Advanced Matrix Extensions)** with 8 2D tile registers on-die for matrix multiplication. 2. **Intel Data Center GPUs (Max Series / Ponte Vecchio):** Built for massive parallelism. Thousands of lightweight vector cores executing millions of threads simultaneously. 3. **Habana Gaudi AI Accelerators (Gaudi 2 & 3):** Purpose-built exclusively for deep learning and LLMs. Featuring 24 Tensor Processor Cores (TPCs) and **on-die 24x 100GbE RoCE RDMA networking (300 GB/s)** directly on silicon!"

- **Whiteboard Table (Draw on board):** | Feature | Intel Xeon CPU | Intel Data Center GPU | Habana Gaudi 2/3 | | :--- | :--- | :--- | :--- | | **Core Type** | Heavy, high-clock cores | Thousands of vector cores | 24 Tensor Processor Cores | | **Special Engine** | Intel AMX (8 KB Tiles) | Xe Matrix Extensions (XMX) | Matrix Math Engine (MME) | | **Best Workload** | OS, scalar math, preprocessing | Dense matrix math, vision | LLM training, Transformers |

☕ 11:35 AM – 11:45 AM: BIO & REJUVENATION BREAK (10 Minutes)

Students stretch, refresh, and get ready for the oneAPI and HPC Toolkit architecture session.

🔷 SLIDE 12: The Software Bridge: Intel oneAPI

- **Time:** 11:45 AM – 12:10 PM (25 Mins) | **Speaker:** Co-Trainer
- **What to Show:** Cross-architecture abstraction layer diagram: Single programming model targeting CPU, GPU, FPGA, and AI Accelerators.
- **Verbatim Trainer Speech:**

> "Before oneAPI, if you wrote GPU code for NVIDIA, you had to use proprietary CUDA. If you ported to Intel or AMD, you had to rewrite everything in OpenCL. **Intel oneAPI solves this through the open Khronos SYCL standard**. You write single-source modern C++ code, and oneAPI compiles it to run across CPUs, GPUs, and FPGAs without vendor lock-in! To initialize oneAPI in any Linux terminal, we run: `source /opt/intel/oneapi/setvars.sh`."

🔷 SLIDE 13: Inside the Intel HPC Toolkit

- **Time:** 12:10 PM – 12:35 PM (25 Mins) | **Speaker:** Co-Trainer + Shivam
- **What to Show:** Base Toolkit + HPC Toolkit layer diagram (`icx` , `icpx` , `ifx` , `Intel MPI` , `Inspector`).
- **Verbatim Trainer Speech:**

> "What is the difference between Base Toolkit and HPC Toolkit? - **Base Toolkit:** Core DPC++/SYCL compiler, oneMKL (math), oneDAL (data analytics), oneDNN (deep learning). - **HPC Toolkit:** Layers directly on top for cluster supercomputing: 1. `icx` & `icpx` (Intel C/C++ Compilers with advanced vectorization). 2. `ifx` (Intel Fortran Compiler for scientific simulations). 3. `Intel MPI Library` (Message passing across cluster nodes). 4. `Intel Inspector` (Memory and thread error debugger)."

🔷 SLIDE 14 & 15: Morning Synthesis & Rapid Check MCQs

- **Time:** 12:35 PM – 12:45 PM (10 Mins) | **Speaker:** Shivam Gupta
- **What to Show:** Pipeline Execution Sequence + 5 Rapid Check Questions.
- **Verbatim Trainer Speech (Shivam):**

> "Let's test our morning foundation with 5 rapid-fire questions before lunch: 1. *Why do laptops fail on 70B parameter models?* (Answer: Memory Wall — lack of memory channels and bandwidth). 2. *What is the dedicated matrix acceleration engine in 4th Gen Xeon?* (Answer: Intel AMX). 3. *What is the open standard behind Intel oneAPI C++?* (Answer: Khronos SYCL). 4. *Which script initializes the oneAPI environment variables?* (Answer: `source /opt/intel/oneapi/setvars.sh`). 5. *Which toolkit provides Intel MPI and cluster compilers?* (Answer: Intel oneAPI HPC Toolkit). Enjoy your lunch! When we return at 01:45 PM sharp, we open our Linux terminals for Session 2: our 3-hour hands-on supercomputing lab!"

🍱 12:45 PM – 01:45 PM: LUNCH BREAK & CLUSTER PREFLIGHT (60 Mins)

- **Trainer Preflight Checklist during break:**
- Confirm all student lab workstation seats have network access.
- Test Google Colab notebook loading: `Day_01_Intel_oneAPI_and_HPC_Toolkit_Master_Lab.ipynb` .
- Verify remote Intel Developer Cloud connectivity (`ssh idc-vm`).

- Ensure projector displays the terminal prompt ready for afternoon hands-on execution.

💻 AFTERNOON HANDS-ON LAB RUNBOOK (SHIVAM'S CORE DOMAIN | 01:45 PM – 04:30 PM)

Lead Trainer: Shivam Gupta | Lab Support: Co-Trainer
Hands-On Lab Reference: `02_DAY_01_HANDS_ON_LAB_AND_COMMANDS.md`
Notebook Reference: `Day_01_Intel_oneAPI_and_HPC_Toolkit_Master_Lab.ipynb`

```
+-------------------------------------------------------------------------------
|                  AFTERNOON HANDS-ON LAB EXECUTION PHASES (3 HOURS)
+-------------------------------------------------------------------------------
| 01:45 PM - 02:30 PM (45m) | Phase 1: Environment Readiness & Silicon Introspection (
| 02:30 PM - 03:15 PM (45m) | Phase 2: Linux Tree Hierarchy, Heredocs & Permissions (S
| 03:15 PM - 04:00 PM (45m) | Phase 3: Automated Shell Scripting — Writing hpc_node_au
| 04:00 PM - 04:30 PM (30m) | Phase 4: SSH Key-Based Remote Access & oneAPI Setup (Sta
| 04:30 PM - 04:50 PM (20m) | Phase 5: 25 In-Class Checkpoint MCQs & 22-Point Checklis
| 04:50 PM - 05:00 PM (10m) | Phase 6: Daily Wrap-Up, Q&A, and Day 2 Preview
+-------------------------------------------------------------------------------
```

🚀 PHASE 1: System Readiness & Silicon Introspection (01:45 PM – 02:30 PM)

Syllabus Mapping: Topics 1 & 2 (Introduction to HPC & Intel Architecture)
Colab Cells: Section 0 & Section 1

Step 1A: What Shivam Says to the Class:

> "Welcome back everyone! Open your terminals or the Day 1 Google Colab notebook right now. Before an HPC engineer runs a single line of parallel code, they must interrogate the physical silicon underneath. We are going to inspect the CPU sockets, hardware cores, cache lines, and vector instruction engines."

Step 1B: Exact Commands Students Run:

```
# 1. Inspect operating system kernel and architecture
uname -sr
whoami
```

```
hostname

# 2. Inspect CPU architecture summary
lscpu

# 3. Extract Core Count, Sockets, and Hardware Threads
lscpu | grep -E "Model name|Socket\(s\)|Core\(s\) per socket|Thread\(s\) per core|NUMA

# 4. Probe for hardware SIMD vector and AMX matrix flags
lscpu | grep -E "avx512|amx"

# 5. Inspect cache hierarchy (L1 data, L1 instruction, L2, and shared L3)
lscpu | grep -E "L1d|L1i|L2|L3"

# 6. Check physical RAM and free memory
free -h
```

Step 1C: Trainer Explanation & Socratic Check:

- Point out the L1d cache (typically 32KB to 48KB per core), L2 cache (1MB to 2MB per core), and L3 shared cache (tens to hundreds of MBs).
- Explain: "*Data inside L1 cache arrives in ~1 nanosecond. Data from DDR5 RAM takes ~65 nanoseconds — over 60 times slower! In HPC, cache-conscious programming is king.*"

📂 PHASE 2: Linux Tree Hierarchy, Heredocs & Permissions (02:30 PM – 03:15 PM)

Syllabus Mapping: Topic 3 (Linux Basics: File system, commands, environment setup)
Colab Cells: Section 2

Step 2A: What Shivam Says to the Class:

> "In supercomputing, messy filesystems cause catastrophic job crashes. If 60 compute nodes cannot find your configuration file because of an incorrect relative path, your million-dollar simulation aborts. We are now going to construct a pristine directory tree, use Heredocs to generate configuration files without GUI editors, and master Linux octal permissions."

Step 2B: Exact Commands Students Run:

```
# 1. Print Current Working Directory
pwd

# 2. Build the standardized HPC workspace hierarchy in one command
mkdir -p ~/hpc_day01/{scripts,data,logs,bin}

# 3. Navigate into scripts directory
cd ~/hpc_day01/scripts
pwd

# 4. Generate cluster configuration file using Heredoc redirection
cat << 'EOF' > cluster_config.env
# ================================================================================
# Intel HPC Cluster Environment Configuration
# ================================================================================
CLUSTER_NAME="GLA_Intel_HPC_Cluster"
HEAD_NODE_IP="10.100.1.1"
COMPUTE_NODES=("node01" "node02" "node03" "node04")
MAX_JOBS=16
DEFAULT_PARTITION="standard"
SCRATCH_STORAGE="/scratch/gla_user"
EOF

# 5. Inspect file creation and verify contents
ls -lh cluster_config.env
cat cluster_config.env

# 6. Search for specific configuration keys using grep
grep "COMPUTE_NODES" cluster_config.env
grep "SCRATCH" cluster_config.env

# 7. Practice absolute vs relative paths
cp cluster_config.env ~/hpc_day01/data/production.env
ls -l ~/hpc_day01/data/

# 8. Environment Variable & PATH Configuration
export GLA_HPC_ROOT="$HOME/hpc_day01"
export PATH="$GLA_HPC_ROOT/bin:$PATH"
echo "Active PATH: $PATH"

# 9. Understand octal permissions
chmod 755 ~/hpc_day01/scripts
ls -ld ~/hpc_day01/scripts
```

Step 2C: Common Pitfall Intervention:

- **The Trailing Space Pitfall:** When setting variables, if a student types `VAR = "val"`, Bash errors with `command not found: VAR`.
  Fix: Remind them: "*Bash uses spaces as argument delimiters! Never put spaces around* `=`."

🤖 PHASE 3: Shell Scripting Automation Drill (03:15 PM – 04:00 PM | 45 Mins)

Syllabus Mapping: Topic 4 (Shell Scripting for Automation)
Colab Cells: Section 2 & Code Directory: `lab_activities/code/`

Step 3A: What Shivam Says to the Class (Sequential Progression Mindset):

> "Now we enter Shell Scripting. We will NOT jump straight into complex 50-line scripts. We will build your confidence sequentially in 4 progressive levels: - **Level 1:** Variables, Shebang, and Command Substitution (`01_basic_syntax_and_vars.sh`). - **Level 2:** Conditionals (`if/else`), Directory Checks, and Exit Codes `$?` (`02_conditions_and_exit_codes.sh`). - **Level 3:** Loops, Arrays, and Cluster Batch Dispatch (`03_batch_loop_automation.sh`). - **Level 4:** Complete Automated Node Hardware & Memory Audit Capstone (`hpc_node_audit.sh`). Master each level before we take the next step!"

Step 3B: Level 1 — Shebang, Variables & Command Substitution (`01_basic_syntax_and_vars.sh`)

```
cat << 'EOF' > ~/hpc_day01/scripts/01_basic_syntax_and_vars.sh
#!/usr/bin/env bash
# RULE: NO SPACES around '=' when assigning variables!
STUDENT_NAME="GLA_Supercomputing_Student"
WORKSHOP_DAY=1
CURRENT_TIME=$(date '+%Y-%m-%d %H:%M:%S')

echo "============================================================"
echo "  STEP 1: BASH BASICS & ENVIRONMENT VARIABLES"
echo "============================================================"
echo "Welcome, ${STUDENT_NAME}!"
echo "Workshop Day     : ${WORKSHOP_DAY}"
echo "Current Hostname : $(hostname)"
echo "Current Time     : ${CURRENT_TIME}"
echo "============================================================"
EOF
```

```
chmod +x ~/hpc_day01/scripts/01_basic_syntax_and_vars.sh
~/hpc_day01/scripts/01_basic_syntax_and_vars.sh
```

Step 3C: Level 2 — Conditionals & Exit Codes (`02_conditions_and_exit_codes.sh`)

```
cat << 'EOF' > ~/hpc_day01/scripts/02_conditions_and_exit_codes.sh
#!/usr/bin/env bash
WORKSPACE_DIR="${HOME}/hpc_day01/data"

echo "============================================================"
echo "  STEP 2: CONDITIONALS & EXIT STATUS CODES"
echo "============================================================"

if [ -d "$WORKSPACE_DIR" ]; then
    echo "[+] Found existing directory: $WORKSPACE_DIR"
else
    echo "[-] Directory not found. Creating it now..."
    mkdir -p "$WORKSPACE_DIR"
    echo "[+] mkdir command completed with exit code: $?"
fi

SAMPLE_FILE="${WORKSPACE_DIR}/sample.txt"
echo "HPC Batch Processing initialized on $(date)" > "$SAMPLE_FILE"

if [ -f "$SAMPLE_FILE" ]; then
    echo "[+] Verified file creation: $SAMPLE_FILE"
    echo "[+] File size: $(wc -c < "$SAMPLE_FILE") bytes"
fi
echo "============================================================"
EOF

chmod +x ~/hpc_day01/scripts/02_conditions_and_exit_codes.sh
~/hpc_day01/scripts/02_conditions_and_exit_codes.sh
```

Step 3D: Level 3 — Loops, Node Arrays & Output Redirection (`03_batch_loop_automation.sh`)

```
cat << 'EOF' > ~/hpc_day01/scripts/03_batch_loop_automation.sh
#!/usr/bin/env bash
COMPUTE_NODES=("node-cpu-01" "node-cpu-02" "node-gpu-01" "node-ai-gaudi")
```

```
LOG_FILE="${HOME}/hpc_day01/logs/cluster_dispatch.log"

> "$LOG_FILE"

echo "============================================================"
echo "  STEP 3: CLUSTER BATCH LOOP & AUTOMATION"
echo "============================================================"
echo "[*] Dispatching tasks across ${#COMPUTE_NODES[@]} compute nodes..."

for node in "${COMPUTE_NODES[@]}"; do
    TIMESTAMP=$(date '+%H:%M:%S')
    echo "[${TIMESTAMP}] Dispatched batch job to ${node} -> Status: ACTIVE" >> "$LOG_F
    echo "  • Node ${node}: Assigned Batch Worker Task"
done

echo "------------------------------------------------------------"
echo "[+] All tasks dispatched! Persisted to: $LOG_FILE"
cat "$LOG_FILE"
echo "============================================================"
EOF

chmod +x ~/hpc_day01/scripts/03_batch_loop_automation.sh
~/hpc_day01/scripts/03_batch_loop_automation.sh
```

Step 3E: Level 4 — Capstone Automated Node Audit Script (`hpc_node_audit.sh`)

```
cat << 'EOF' > ~/hpc_day01/scripts/hpc_node_audit.sh
#!/usr/bin/env bash
# ================================================================================
# Script Name : hpc_node_audit.sh
# Purpose     : Automated Compute Node Hardware & Software Health Audit
# Author      : GLA University HPC Masterclass (Lead: Shivam Gupta)
# ================================================================================

set -euo pipefail

LOG_DIR="${HOME}/hpc_day01/logs"
mkdir -p "$LOG_DIR"
TIMESTAMP="$(date '+%Y%m%d_%H%M%S')"
LOG_FILE="${LOG_DIR}/node_audit_${TIMESTAMP}.log"

{
    echo "============================================================"
```

```
    echo "       INTEL HPC NODE HARDWARE & SOFTWARE AUDIT           "
    echo "==========================================================="
    echo "Timestamp          : $(date '+%Y-%m-%d %H:%M:%S')"
    echo "Host Node Name     : $(hostname)"
    echo "Active User        : $(whoami)"
    echo "Linux OS Kernel    : $(uname -r)"
    echo "Architecture       : $(uname -m)"
    echo "CPU Model          : $(lscpu | grep 'Model name:' | head -n1 | sed 's/Model
    echo "Available Cores    : $(nproc)"
    echo "Total Memory (RAM) : $(free -h | awk '/^Mem:/ {print $2}')"
    echo "Available Memory   : $(free -h | awk '/^Mem:/ {print $7}')"
    echo "Filesystem / Free  : $(df -h / | awk 'NR==2 {print $4}')"
    echo "-----------------------------------------------------------"
    echo "Hardware Acceleration Probing:"

    if grep -q "avx512" /proc/cpuinfo 2>/dev/null; then
        printf "  %-20s: [SUPPORTED]\n" "AVX-512 Vector"
    else
        printf "  %-20s: [NOT DETECTED]\n" "AVX-512 Vector"
    fi

    if grep -q "amx" /proc/cpuinfo 2>/dev/null; then
        printf "  %-20s: [SUPPORTED]\n" "Intel AMX Matrix"
    else
        printf "  %-20s: [NOT DETECTED]\n" "Intel AMX Matrix"
    fi

    echo "-----------------------------------------------------------"
    echo "Intel oneAPI & Development Tools In PATH:"
    for tool in bash ssh icpx ifx mpirun sycl-ls g++; do
        if command -v "$tool" >/dev/null 2>&1; then
            printf "  %-12s: AVAILABLE (%s)\n" "$tool" "$(command -v "$tool")"
        else
            printf "  %-12s: NOT FOUND IN PATH\n" "$tool"
        fi
    done
    echo "==========================================================="
} | tee "$LOG_FILE"

echo ""
echo "[INFO] Audit successfully completed and persisted to: $LOG_FILE"
EOF

chmod +x ~/hpc_day01/scripts/hpc_node_audit.sh
```

```
~/hpc_day01/scripts/hpc_node_audit.sh
ls -la ~/hpc_day01/logs/
```

Step 3F: Walkabout Verification:

- Co-Trainer and Shivam walk the aisles. Verify all 60 students have run Levels 1 through 4 and see their persistent audit logs!

🔒 PHASE 4: SSH Key Generation & Remote Access (Dual-Scenario Protocol) (04:00 PM – 04:30 PM | 30 Mins)

Syllabus Mapping: Topics 5, 6 & 7 (SSH, oneAPI Toolkit, Intel HPC Toolkit)
Colab Cells: Sections 4, 5, 9

Step 4A: What Shivam Says to the Class:

> "Now Phase 4: Remote Supercomputing Access. Supercomputers are never accessed with passwords — passwords are slow, brute-forceable, and fail in automated batch scripts. We generate an industrial-grade **Ed25519** elliptic curve key pair. Notice: `id_ed25519` is your Private Key (permissions strictly `600` — never share!). `id_ed25519.pub` is your Public Key (safe to share with the server). We will demonstrate both scenarios: connecting to a live cluster, and verifying the cryptographic handshake on your own machine!"

Step 4B: Hands-On Key Generation on Student Machines:

```
# 1. Create secure .ssh folder with strict 700 permissions
mkdir -p ~/.ssh
chmod 700 ~/.ssh

# 2. Generate secure Ed25519 SSH keypair
ssh-keygen -t ed25519 -C "student@gla_hpc_cluster" -f ~/.ssh/id_ed25519_gla -N ""

# 3. Verify key generation and strict private key permissions (600)
ls -la ~/.ssh/id_ed25519_gla*
chmod 600 ~/.ssh/id_ed25519_gla
chmod 644 ~/.ssh/id_ed25519_gla.pub
```

```
# 4. View public key
cat ~/.ssh/id_ed25519_gla.pub
```

Step 4C: Dual-Scenario Execution:

- **SCENARIO A (Remote Server is Available in Lab):** `bash # Copy public key to server: ssh-copy-id -i ~/.ssh/id_ed25519_gla.pub username@server_ip # Log in passwordless: ssh -i ~/.ssh/id_ed25519_gla username@server_ip -p 22`
- **SCENARIO B (Zero-Server Mode — Local Loopback & Self-Authorization):** *(If no external server IP is provided, students authorize the key locally on their own workstation)* `bash # Authorize key locally: cat ~/.ssh/id_ed25519_gla.pub >> ~/.ssh/authorized_keys chmod 600 ~/.ssh/authorized_keys grep "student@gla_hpc" ~/.ssh/authorized_keys echo "[+] SSH Key Pair successfully configured and authorized for passwordless access!"`

Step 4D: Live Trainer Projector Demo (Shivam):

- Shivam shows live connection to the enterprise Intel cluster: `bash ssh idc-vm lscpu | grep -E "Model name|avx512|amx"`
- Show the class the genuine 4th Gen Intel Xeon Gold 6442Y (Sapphire Rapids) with active `amx_tile` and `avx512_bf16` flags!

Step 4E: Initialize oneAPI Environment:

```
source /opt/intel/oneapi/setvars.sh 2>/dev/null || source /opt/intel/oneapi/oneapi-var
sycl-ls 2>/dev/null || echo "[i] SYCL device query completed."
```

🎯 PHASE 5: 25 In-Class Checkpoint MCQs & Checklist Sign-Off (04:30 PM – 04:50 PM)

Syllabus Mapping: All Modules 1 through 7
Source File: `GLA_UNIVERSITY_DAY_01_IN_CLASS_CHECKPOINT_MCQS_OFFICIAL.md`
Student Paper: `GLA_UNIVERSITY_DAY_01_IN_CLASS_CHECKPOINT_STUDENT_PAPER.md`

Top 5 Critical In-Class MCQs to Discuss Aloud:

1. **Q01 (Memory Wall):** Why do laptops fail on 70B LLMs?
   Answer: `(a)` Choked by Memory Wall (2 DDR channels vs 16 DDR5 channels on Xeon).
2. **Q02 (Flynn's Taxonomy):** What model describes modern GPUs and AVX-512?
   Answer: `(b)` SIMD (Single Instruction, Multiple Data).
3. **Q03 (Amdahl's Law):** If sequential fraction is 10%, what is max theoretical speedup?
   Answer: `(b)` 10x ($1 / 0.10$).
4. **Q05 (Bash Redirection):** What does `set -euo pipefail` do?
   Answer: Aborts script immediately on error, unbound variable, or pipe failure.
5. **Q10 (oneAPI Activation):** Which command activates oneAPI compilers in shell?
   Answer: `source /opt/intel/oneapi/setvars.sh`.

The 22-Point Mastery Checklist Sign-Off:

Project the 22-Point Checklist from `03_DAY_01_STUDENT_CHEATSHEET_AND_MCQS.md` : - [x] 01. HPC concept & purpose - [x] 02. HPC system overview (Head vs Compute nodes) - [x] 03. HPC vs ordinary computing (Memory Wall) - [x] 04. Intel Xeon Scalable Processors (Sapphire Rapids, AMX) - [x] 05. Intel Data Center GPUs (Max Series / Ponte Vecchio) - [x] 06. Habana Gaudi AI Accelerators (Gaudi 2/3, 24x 100GbE RoCE) - [x] 07. Silicon hardware roles in HPC - [x] 08. Linux filesystem hierarchy & paths (Absolute vs Relative) - [x] 09. Core Linux navigation & file commands (`ls` , `cd` , `mkdir` , `cat` , `grep`) - [x] 10. Linux environment variables & `$PATH` - [x] 11. Shell script creation & execution permissions (`chmod +x`) - [x] 12. Shell variables and command execution - [x] 13. Shell task automation workflows (`hpc_node_audit.sh`) - [x] 14. SSH remote access purpose - [x] 15. SSH connection syntax & ports (Port 22) - [x] 16. Password vs Key-based authentication (`ed25519` , `600` permissions) - [x] 17. Intel oneAPI Toolkit purpose - [x] 18. oneAPI components overview (DPC++/SYCL, oneMKL, oneDNN) - [x] 19. oneAPI environment access (`setvars.sh` , `sycl-ls`) - [x] 20. Intel HPC Toolkit introduction - [x] 21. HPC compilation workflow (`icx` , `icpx` , `ifx`) - [x] 22. Intel HPC Toolkit tools (Intel MPI, Inspector, ITAC)

👷 PHASE 6: Daily Wrap-Up & Day 2 Teaser (04:50 PM – 05:00 PM)

- **What Shivam Says to the Class:**

> "Outstanding work today! Every single one of you has built an unbreakable supercomputing foundation. You understand the silicon physics of Xeon CPUs, GPUs, and Gaudi accelerators. You navigated Linux, automated node health audits using Bash, generated cryptographic SSH keys, and initialized the Intel oneAPI ecosystem. Tomorrow on Day 2, we break the serial bottleneck! We will dive into **OpenMP multi-threaded shared-memory parallelism**, write **MPI distributed message-passing applications**, and

> accelerate classical machine learning by 20x using **Intel oneDAL and the Intel AI Analytics Toolkit**. Please be seated by 09:30 AM tomorrow. Have a great evening!"

🛠️ TOP 5 STUDENT GOTCHAS & 30-SECOND RAPID INTERVENTIONS

#	Error / Symptom	Root Cause	30-Second Trainer Fix
1	`bash: ./script.sh: Permission denied`	Created file lacks executable bit	Run `chmod +x script.sh` (or `chmod 755 script.sh`).
2	`command not found: VAR="value"`	Student placed spaces around `=`	Explain: Bash interprets spaces as arguments. Use `VAR="value"`.
3	`cd: No such file or directory`	Student used relative path from wrong folder	Run `pwd` first to check CWD, then use absolute path `cd ~/hpc_day01/scripts`.
4	`Permissions 0777 for 'id_ed25519' are too open`	SSH client refuses private key with loose permissions	Run `chmod 600 ~/.ssh/id_ed25519`. Private keys must be owner-only.
5	`icpx: command not found`	oneAPI environment variables not active	Run `source /opt/intel/oneapi/setvars.sh` in the current terminal.

📚 MASTER CURRICULUM DIRECTORY & REFERENCE LOCATOR

All paths are 100% active and verified in the workspace:

Asset Type	File Name	Description & Purpose
Official Syllabus	`extracted_hpc_pdf.txt`	Official GLA University HPC curriculum (Page 2)

Asset Type	File Name	Description & Purpose
Master Presentation	`Day-01_Master_Presentation.pptx`	Official 15-Slide Master Deck (Edulateral / GLA)
Slide Speaker Notes	`DAY_01_MASTER_PRESENTATION_AND_SPEAKER_NOTES.md`	Slide-by-slide speaker notes, scripts & visual cues
OCR Transcript	`Day_01_Slide_OCR_Transcript.md`	Exact OCR text of all 15 slides
Colab Notebook	`Day_01_Intel_oneAPI_and_HPC_Toolkit_Master_Lab.ipynb`	Complete Google Colab hands-on master lab
Trainer Playbook	`01_DAY_01_6_HOUR_MASTER_TRAINER_PLAYBOOK.md`	6-Hour Master Trainer Playbook
Hands-On Runbook	`02_DAY_01_HANDS_ON_LAB_AND_COMMANDS.md`	8-Stage practical command execution runbook
Student Cheatsheet	`03_DAY_01_STUDENT_CHEATSHEET_AND_MCQS.md`	1-Page Cheatsheet + 22 Checklist Points + 50 MCQs
Topic Study Guide	`04_DAY_01_COMPLETE_TOPIC_BY_TOPIC_STUDY_GUIDE.md`	Exhaustive concept &

Asset Type	File Name	Description & Purpose
		technical notes
Visual Arrow Trees	`05_DAY_01_ARROW_WORKFLOWS_AND_TOPIC_TREES.md`	Flowcharts: Concept ──► Silicon ──► Command ──► Verify
Concept Glossary	`00_DAY_01_MASTER_TERMINOLOGY_AND_CONCEPT_GLOSSARY.md`	Master glossary of all HPC terms
Pre-Assessment (Ans)	`GLA_UNIVERSITY_DAY_01_50_MCQ_PRE_ASSESSMENT_OFFICIAL.md`	50 Pre-Assessment MCQs with official answer keys
Pre-Assessment (Paper)	`GLA_UNIVERSITY_DAY_01_PRE_ASSESSMENT_STUDENT_QUESTION_PAPER.md`	Student Question Paper (Questions only)
In-Class Checkpoints	`GLA_UNIVERSITY_DAY_01_IN_CLASS_CHECKPOINT_MCQS_OFFICIAL.md`	25 Socratic In-Class Checkpoint MCQs with answers
Post-Assessment (Ans)	`GLA_UNIVERSITY_DAY_01_50_MCQ_POST_ASSESSMENT_OFFICIAL.md`	50 Post-Assessment MCQs with official answer keys
Shell Audit Script	`hpc_node_audit.sh`	Automated hardware audit script

Asset Type	File Name	Description & Purpose
oneAPI Setup Script	`setup_oneapi_env.sh`	oneAPI environment sourcing script